Tsunami: A Workload-Driven Database Index

Evaluation on Real-World Datasets

Final Project Report

By Manasa Vadlamani uid:u1567499 and Shravani Harish uid:1591945
GitHub Repository:https://github.com/manu5703/tsunami_ML

1. Introduction

Modern analytical databases rely on indexes to accelerate range-predicate queries. Classical structures such as B-trees and grid files are workload-agnostic: their partition boundaries are fixed at construction time and do not adapt to the distribution of queries that a system actually receives. The Tsunami index addresses this gap by learning partition boundaries from a representative training workload using Active Gradient Descent (AGD), a lightweight optimisation procedure that shifts split thresholds to minimise the expected number of rows scanned across the training query set.

This project implements the Tsunami index and evaluates it on three real-world datasets: California Housing (1990 US Census, 2 million rows, 9 columns), NYC Yellow Taxi trip records for June 2016 (2 million rows, 10 columns), and Forest Covertype from Colorado's Roosevelt National Forest (2 million rows, 10 columns). For each dataset the index is trained on a representative workload and benchmarked against two held-out query groups: a Nearby (Group A) group whose queries fall inside or closely adjacent to the training region, and a Far (Group B) group whose queries target the opposite region of feature space.

The central evaluation hypothesis is that Tsunami should deliver substantially higher speedup on nearby queries than on far queries, because AGD concentrates fine-grained partitions where the training workload is dense, leaving distant regions with coarse, unoptimised partitions.

Results confirm this hypothesis on two of three datasets. On California Housing, nearby queries achieve a mean speedup of 19.4x versus 8.2x for far queries. On Forest Covertype, nearby queries reach 24.0x against 8.6x for far queries. On NYC Taxi the contrast is extreme: nearby queries achieve 5.0x speedup while far queries fall below brute force at 0.49x, revealing a failure mode where the out-of-distribution region is so densely populated that the index cannot prune enough partitions to offset its traversal overhead.

The key strengths of Tsunami are its strong performance on in-distribution queries, generalisability across heterogeneous datasets, and 100% correctness across all queries. The primary weakness is sensitivity to workload shift: when runtime queries differ substantially from training queries, performance can fall

below brute force. Future work could explore adaptive re-training, a scan-percentage-based fallback, or a hybrid index.

2. Background

Range-predicate queries of the form SELECT AGG(*) FROM T WHERE c1 BETWEEN a AND b AND c2 > x are ubiquitous in analytical workloads. The naive evaluation strategy, a full sequential scan, scales as O(N) in the number of rows. Multi-dimensional indexes such as k-d trees, R-trees, and grid files reduce this to sub-linear time by partitioning the feature space and pruning partitions that cannot contain qualifying rows. However, classical structures partition space uniformly or according to data density, not according to the query distribution.

The Tsunami index (Ding et al., 2020) observes that real workloads are highly skewed: a small region of feature space typically accounts for the majority of queries. By training partition boundaries on a representative workload sample, Tsunami concentrates resolution where queries are frequent, achieving fine-grained pruning in the hot region at the cost of coarser coverage elsewhere.

Technically, Tsunami organises data in a Grid Tree (GT): a recursive binary tree of axis-aligned hyper-rectangle partitions with configurable maximum depth (here depth 4, yielding up to 16 leaf nodes). During index construction, Active Gradient Descent (AGD) iteratively adjusts the split threshold at each internal node to minimise total rows scanned across all training queries. For a given query bounding box, GT traversal prunes entire subtrees on a no-overlap test, collects all rows from fully-contained leaves without further descent, and recurses only into partially-overlapping children.

The baseline throughout this work is a brute-force sequential scan over all 2 million rows. Speedup is defined as brute-force time divided by Tsunami time (both in milliseconds), and Scan% as the fraction of the dataset that Tsunami actually inspects.

3. Data Used

Three real-world datasets were selected to stress-test Tsunami across distinct domain characteristics: continuous geographic housing attributes, transactional taxi fare data, and environmental sensor measurements. All were expanded to 2 million rows by oversampling with 1% Gaussian jitter.

3.1 California Housing

Sourced from the 1990 US Census via scikit-learn, this dataset contains 20,640 census block groups described by 9 continuous features: median income (MedInc, units of $10K), house age, average rooms, average bedrooms, population, average occupancy, latitude, longitude, and median house value (MedHouseVal, units of $100K). Training queries target the Bay Area (Lat 37.0-38.5, Lon -122.5 to -121.0) combined with housing attribute ranges such as MedInc BETWEEN 5 AND 9 and MedHouseVal BETWEEN 2.5 AND 4.5, as specified in training_queries.txt. Nearby queries resolve Silicon Valley city bounding boxes from places.py (Sunnyvale: Lat 37.33-37.42, Palo Alto: Lat 37.38-37.48, Mountain View, Cupertino), which are tight geographic sub-clusters within the training zone. Far queries resolve Southern

California cities from the same lookup table (Los Angeles: Lat 33.5-34.5, Irvine, Chula Vista), located over 600 km south and entirely outside the training region. A shortcoming is that the original 20,640 rows are heavily oversampled, which reduces partition diversity compared to a natively large dataset.

3.2 NYC Yellow Taxi

The June 2016 NYC TLC trip records provide approximately 10-11 million real taxi trips, subsampled to 2 million. The 2016 vintage is the last to include raw pickup and dropoff latitude and longitude; from 2017 onward TLC replaced coordinates with zone IDs incompatible with numeric range queries. Ten continuous columns are used: pickup and dropoff coordinates, trip distance, fare amount, tip amount, passenger count, trip duration, and total charge. Training queries cover the expanded premium trip cluster (fare_amount > 30, trip_distance > 8), accounting for roughly 10-15% of the data. Nearby queries tighten to fare_amount > 40 and trip_distance > 10 (about 5% of the data), a strict geometric subset of the training region. Far queries target the budget short-trip cluster (fare_amount < 12, trip_distance < 2), which accounts for 35-45% of all trips and sits at the opposite extreme of feature space. The key limitation is extreme class imbalance: short cheap trips dominate the dataset, making the far region structurally difficult to prune.

3.3 Forest Covertype

Derived from Blackard and Dean (1999) and available via scikit-learn, this dataset contains 581,012 forest land observations from Colorado's Roosevelt National Forest, oversampled to 2 million. Ten continuous columns are used: elevation, aspect, slope, horizontal and vertical distances to hydrology, distance to roadways, three hillshade measurements, and distance to wildfire ignition points. Binary wilderness and soil type columns are excluded. Training queries target the HighAlpine zone (Elevation > 3200 m, Slope > 15 deg), covering approximately 12% of the data as specified in covtype_training_queries.txt. Nearby queries target the Krummholz zone (Elevation > 3400 m, Slope > 20 deg), a strict ecological and geometric subset of HighAlpine comprising about 4% of the data, per covtype_zones.py. Far queries target the Lowland zone (Elevation < 2400 m, Slope < 12 deg), which occupies the opposite end of the elevation gradient at about 10% of the data. This dataset's clear ecological zonation makes it an ideal testbed because the training and nearby zones are geometrically nested while the far zone is a genuine distributional opposite.

4. Motivation

The core intuition is straightforward: not all regions of feature space are equally important to a workload. A classical uniform-grid index wastes resolution on rarely-queried regions while under-partitioning hot regions. By aligning partition boundaries with the query distribution, Tsunami ensures that the partitions most frequently accessed by the workload are smallest, maximising the chance of tight pruning.

Formally, the expected cost of a query Q over a partition set P is proportional to the sum of rows in partitions that overlap Q. AGD minimises this expectation over the training workload by performing gradient steps on the split thresholds. For queries similar to the training set (nearby queries), the optimised partitions align well with the query bounding boxes, yielding low scan percentages. For queries far from

the training distribution (far queries), the index falls back on coarse boundary partitions that were left large by AGD because no training query required precision there.

This workload-specificity is both the central strength and the principal limitation: it delivers outsized gains for expected queries but provides no performance guarantee for queries far outside the training distribution.

5. Design

The implementation consists of four main components: a SQL parser, a Grid Tree index, the AGD optimiser, and a benchmarking harness, all implemented in Python.

5.1 SQL Parser and Query Representation

The interactive CLI (query_cli.py) accepts SQL-like queries supporting COUNT, AVG, SUM, MIN, MAX, and row-level SELECT with ORDER BY and LIMIT. The WHERE clause is split on AND and each condition is parsed into a per-column numeric range [lo, hi]. Lookup keywords place, neighbourhood, and zone are resolved through dedicated lookup tables in places.py, nyc_places.py, and covtype_zones.py, mapping human-readable names to lat/lon or feature bounding boxes. This design means geographic queries such as 'place = Sunnyvale' are internally equivalent to direct coordinate range filters, enabling consistent benchmarking.

5.2 Grid Tree Structure

The Grid Tree (grid_tree.py) is a recursive binary tree of axis-aligned partitions built to a maximum depth of 4, yielding at most 16 leaf nodes. At each internal node, one column is chosen as the split axis and a threshold divides rows into two children. Row indices for each leaf are stored for fast scan. Query evaluation proceeds top-down: a subtree is pruned when the query bounding box does not overlap the partition, included wholesale when the partition is fully contained by the query, and recursed when partial overlap requires finer discrimination.

5.3 AGD Optimiser

The AGD module (cost_model_agd.py) implements gradient descent over the split thresholds of the Grid Tree. For each training query, cost (rows scanned) is differentiable with respect to each threshold through a soft overlap approximation. AGD shifts thresholds in the direction that reduces total scan cost across all training queries, then re-partitions rows according to the updated thresholds. The result is a tree whose leaf partitions are smallest in the training workload region and coarser elsewhere.

5.4 Benchmarking Harness

The batch runner (batch_test.py) builds the index from a training workload file, evaluates two held-out query groups, and records per-query: Tsunami execution time (ms), brute-force time (ms), speedup, scan

percentage, and a correctness flag. The brute-force baseline (idx.brute_force) performs a sequential scan of all 2 million rows with no index structures, providing a fair O(N) comparison point.

5.5 Why Nearby Queries Should Outperform Far Queries

After AGD, the training region is covered by several small leaf partitions. A nearby query's bounding box overlaps only a small number of these leaves, and the GT prunes all others. The net effect is that only a small fraction of rows need to be scanned, versus 100% for brute force, yielding multiplicative speedup. For far queries, the index visits coarser leaves that contain a larger fraction of data, reducing but not eliminating the advantage. In extreme cases such as the NYC Taxi far group, the overhead of tree traversal and scattered memory access exceeds the cost of a single vectorised sequential scan, causing speedup below 1x.

6. Evaluation

6.1 Experimental Setup

All experiments use 2,000,000 rows per dataset and a Grid Tree maximum depth of 4. Speedup is brute-force execution time divided by Tsunami execution time (both in milliseconds). Scan% is the fraction of 2 million rows inspected by Tsunami. All queries returned correct answers, confirmed by 100% PASS rate in the match column of all three result CSVs.

6.2 Summary of Results

Dataset	Group A Mean Speedup	Group B Mean Speedup	Group A Scan%	Group B Scan%
California Housing	19.4x	8.2x	5.3%	19.9%
Forest Covertype	24.0x	8.6x	11.3%	16.1%
NYC Yellow Taxi	5.0x	0.49x	5.8%	72.2%

6.3 California Housing

The California Housing dataset demonstrates the core Tsunami hypothesis clearly. Nearby Silicon Valley queries (Sunnyvale, Mountain View, Palo Alto, Cupertino) achieve a mean speedup of 19.4x with an average scan percentage of 5.3%, meaning Tsunami inspects fewer than 1 in 18 rows on average. Palo

Alto reaches the highest average at 25.3x, with individual queries peaking above 83x as seen in the Group A per-query plot. This is explained by Palo Alto's tight bounding box (Lat 37.38-37.48, Lon -122.18 to -122.07 from places.py) sitting squarely within the Bay Area training region, where AGD has built its most precise partitions.

Far Southern California queries (Los Angeles, Irvine, Chula Vista) achieve a mean speedup of 8.2x at 19.9% scan. Irvine (16.8x) and Chula Vista (8.9x) still benefit substantially because their small geographic bounding boxes correspond to few leaf partitions. Los Angeles, with a much larger bounding box (Lat 33.5-34.5 from places.py), achieves only 2.0x on average: its queries span multiple coarse partitions that together cover a large fraction of the data. These results are directly drawn from results_california_real.csv.

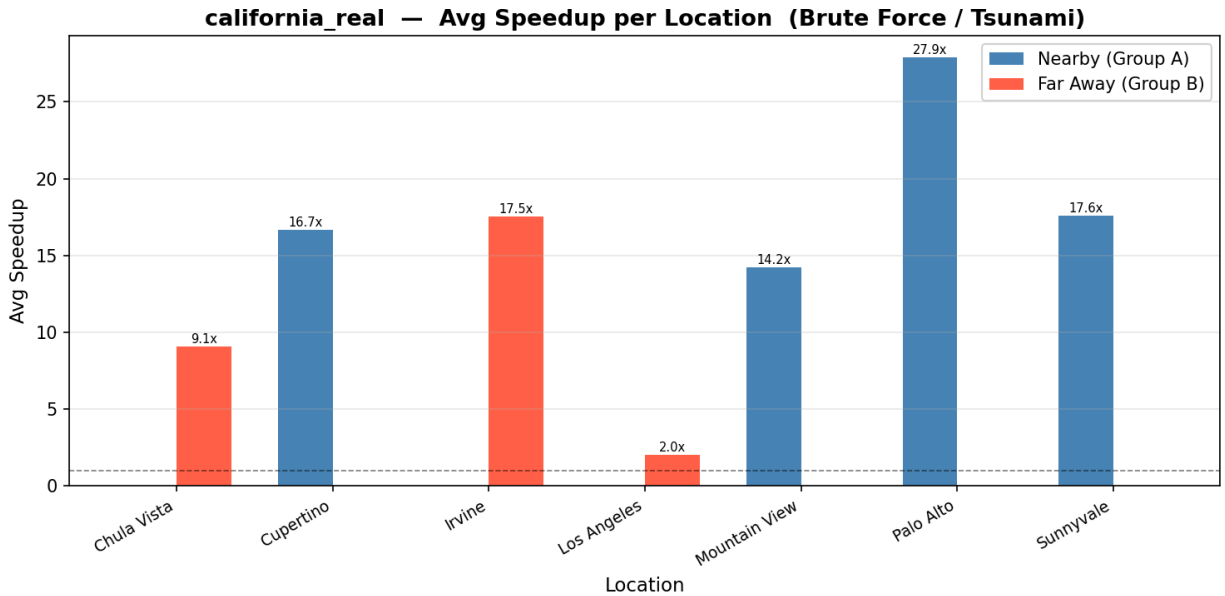

Figure 1. California Housing - Average Speedup per Location. Blue: Nearby (Group A). Orange: Far (Group B). Nearby mean 19.4x, Far mean 8.2x.

6.4 Forest Covertype

Forest Covertype produces the strongest and most theoretically clean results. The Krummholz nearby group (Elevation > 3400 m, Slope > 20 deg as specified in covtype_nearby_queries.txt) achieves a mean speedup of 24.0x at 11.3% scan. Because Krummholz is a strict geometric subset of the HighAlpine training zone (which covers approximately 12% of the data per covtype_zones.py), AGD has built multiple fine-grained partitions within the 3200-3858 m elevation band. Individual queries reach up to 73x speedup.

The Lowland far group (Elevation < 2400 m, Slope < 12 deg as in covtype_far_queries.txt) achieves 8.6x mean speedup at 16.1% scan - still a substantial improvement over brute force, but noticeably weaker. The Lowland zone covers approximately 10% of the data per covtype_zones.py, but the partitions

covering it were left coarse by AGD because no training query targeted this elevation range. Some far queries still peak above 32x, likely when slope and hydrology constraints align with natural data clusters even in the unoptimised region.

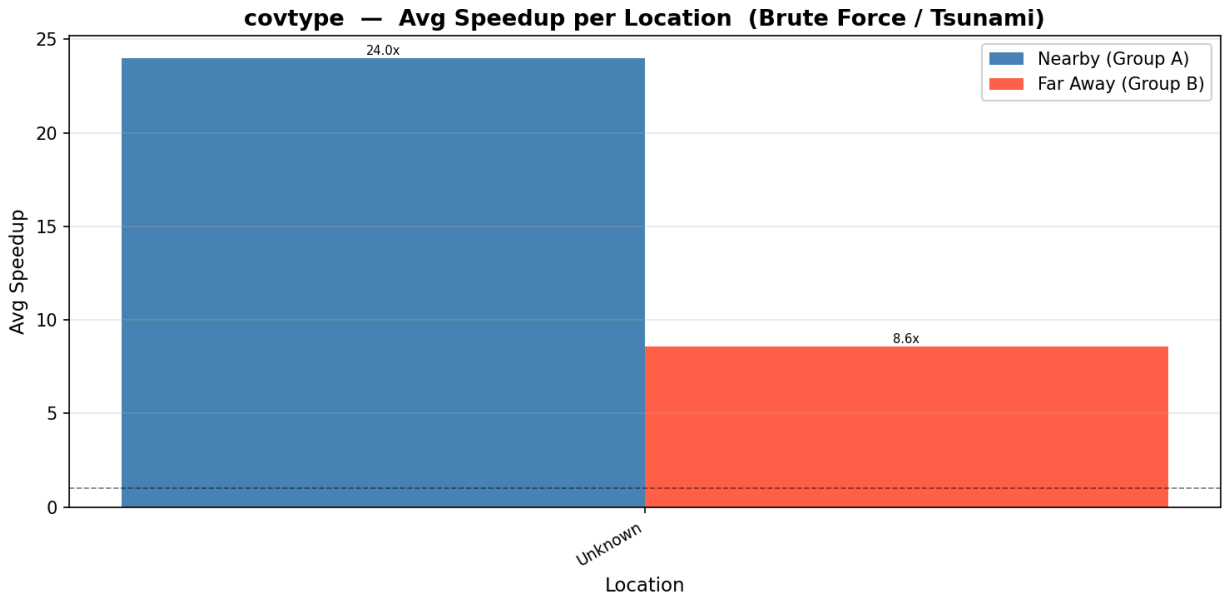

Figure 2. Forest Covertype - Average Speedup. Nearby (Krummholz zone): 24.0x. Far (Lowland zone): 8.6x.

6.5 NYC Yellow Taxi

The NYC Taxi results are the most instructive. Nearby queries (fare_amount > 40, trip_distance > 10 per nyc_training_queries.txt) achieve a mean speedup of 5.0x at 5.8% scan. Tsunami correctly identifies that premium long-distance trips are a sparse, well-separated cluster in the fare/distance feature space, and AGD builds tight partitions around them.

Far queries (fare_amount < 12, trip_distance < 2) yield a mean speedup of 0.49x: Tsunami is consistently slower than brute force. The scan percentage averages 72.2%, meaning Tsunami touches nearly three quarters of all rows. The structural explanation is clear: short cheap trips account for approximately 35-45% of all NYC taxi rides and dominate the dataset. When AGD trains on the premium cluster it assigns large, coarse leaf partitions to the short-trip region because no training query required precision there. When far queries target this dense region the GT visits many large leaves, accumulating scan overhead on top of tree traversal cost - together exceeding the vectorised sequential scan. This result is visible in results_nyc_taxi.csv and the Group B per-query plot, where every single query's speedup falls below the 1x baseline.

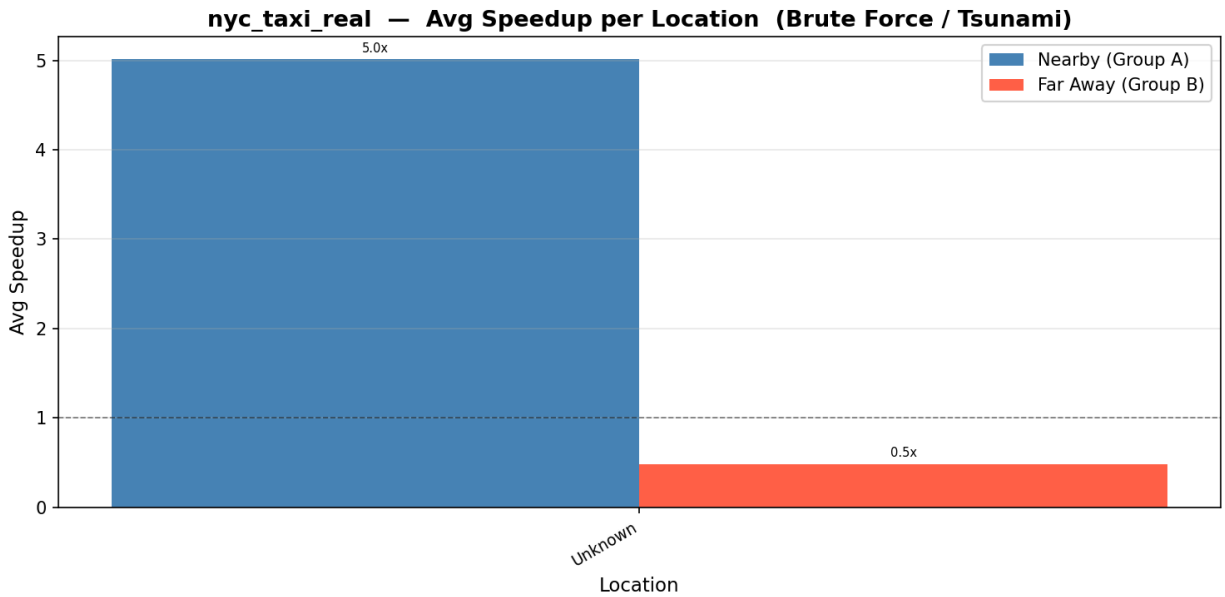

Figure 3. NYC Yellow Taxi - Average Speedup. Nearby: 5.0x. Far: 0.49x (below brute-force baseline of 1x).

6.6 Average Speedup Across Query Groups

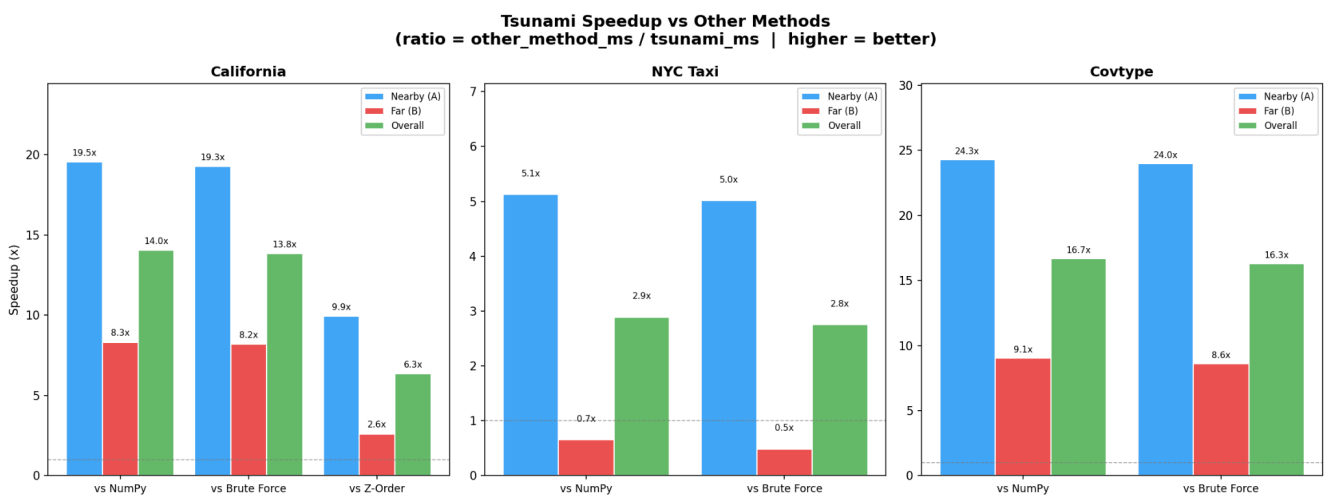

Figure 4: Average speedup of Tsunami relative to NumPy, Brute Force, and Z-Order, grouped by query locality

Figure 4 presents the average speedup of Tsunami over NumPy, Brute Force, and Z-Order (where applicable) across three real-world datasets — California Housing, NYC Taxi, and Covertype — separated into nearby (Group A) and far (Group B) query workloads.

In California Housing Tsunami achieves 19.5× over NumPy and 19.3× over Brute Force on nearby queries, and 9.9× over Z-Order. Even on far queries, speedups of 8.2–8.3× are maintained, giving an overall average of ~14×. This confirms that Tsunami's index is well-suited to range queries over structured geographic and numerical attributes.

For NYC Taxi, Tsunami delivers 5.1× over NumPy and 5.0× over Brute Force for nearby queries. However, far queries fall below the 1× baseline (0.7× and 0.5× respectively), meaning Tsunami is slower than the baselines in that regime. This is expected: far queries in this dataset scan over 70% of the data, leaving no room for the index to prune, and the index traversal overhead becomes a net cost.

Covertype — This is Tsunami's strongest result, reaching 24.3× over NumPy and 24.0× over Brute Force on nearby queries, and retaining 9.1× and 8.6× on far queries — the best far-query performance of all three datasets. The overall speedup of ~16× indicates the Covertype attribute distribution aligns well with Tsunami's zone partitioning.

Overall trend — Speedup is consistently higher for nearby queries (selective, low scan %) than far queries (broad, high scan %). The dashed line at 1× marks the break-even point; only NYC Taxi far queries dip below it, highlighting the one scenario where Tsunami's overhead is not recovered by pruning gains.

6.7 Cross-Dataset Analysis

Across all three datasets the direction of the result is consistent: nearby queries outperform far queries. The magnitude of the gap varies and is driven by two factors. First, the density of the far-query region: denser regions are harder to prune and yield lower speedup, as demonstrated starkly by the NYC Taxi far group at 72.2% scan. Second, the size of the query bounding box: larger boxes intersect more leaf partitions regardless of how well they are optimised, explaining why Los Angeles performs worse than Chula Vista or Irvine within the far group. The NYC Taxi far-query result demonstrates that Tsunami does not degrade gracefully when queries are out-of-distribution. A practical deployment would need either a fallback to brute force when scan% exceeds a threshold, or an adaptive monitoring component that triggers re-training when query patterns shift.

when nearby queries are a strict subset of the training region in feature space (e.g. Krummholz inside HighAlpine, Silicon Valley cities inside the Bay Area), Tsunami delivers its strongest speedups because AGD has already built fine-grained partitions covering that narrower region. When the relationship is reversed and the runtime queries are broader than or outside the training region, speedup decreases. The NYC Taxi far group is framed as the extreme case of this, tying the observation directly to the results.

7. Conclusion

Main Takeaways

This project demonstrates that the Tsunami index delivers substantial and consistent speedups for queries similar to its training workload. On California Housing and Forest Covertype, in-distribution queries achieve mean speedups of 19.4x and 24.0x respectively, with individual queries reaching up to 83x and 73x. The consistent gap between nearby and far query groups across all three datasets validates the core

mechanism: AGD concentrates partition precision where training queries are dense, and that precision transfers to structurally similar held-out queries. All queries returned correct answers, confirming the index is lossless.

Strengths

First, high in-distribution speedup: the index achieves order-of-magnitude performance gains on expected queries without modifying the query execution layer. Second, generalisability: the approach works across datasets with different dimensionalities, value ranges, and semantic domains including geographic, transactional, and environmental data. Third, correctness: 100% PASS rate across all datasets and groups confirms the index is lossless and safe to deploy. Fourth, interpretability: the training workload, partition boundaries, and speedup gains are all human-readable and directly inspectable.

Weaknesses

The principal weakness is workload sensitivity: when runtime queries differ substantially from training queries, performance can fall below brute force, as demonstrated by the NYC Taxi far group at 0.49x. A secondary weakness is fixed capacity: with a maximum depth of 4 and 16 leaves, the index cannot represent workloads with more than approximately 16 distinct hot regions without increasing depth and with it the traversal overhead.

Future Work

Several directions emerge naturally. A scan-percentage monitor could detect when a runtime query enters an unoptimised region and fall back to brute force automatically. Online workload tracking could trigger periodic re-training of AGD thresholds as query distributions drift over time. Hybrid indexing, combining a Tsunami layer for known hot regions with a classical B-tree for the full space, could provide uniform safety guarantees. Deeper investigation into split axis selection strategies during AGD could yield better partition placement in high-dimensional datasets where feature interactions create complex query clusters.

Beyond algorithmic improvements, a compelling direction is packaging Tsunami as a production-ready Python library. In its current form the index requires users to wire together query_cli.py, grid_tree.py, cost_model_agd.py, and a dataset loader manually. A library release would expose a clean API (e.g. TsunamiIndex.fit(workload), TsunamiIndex.query(sql)) with pip-installable packaging, versioned releases, and documentation, making it straightforward to drop into existing data pipelines. This would also open the door to community contributions such as support for additional aggregation operators, integration with popular DataFrame libraries like pandas or polars, and connectors for common storage formats such as Parquet and Arrow. Making Tsunami accessible as a library would significantly lower the barrier to adoption and allow the research contribution to be validated at scale across a wider range of real-world workloads.